Manual de rutas

1. Convenciones de este manual

La API (FastAPI) atiende por defecto en `http://localhost:8000`; su documentación interactiva está en `/docs`. El frontend la consume con las funciones de `front/api_client.py`, que usan la variable de entorno `NEXO_API_URL` o, si no existe, esa dirección. Las peticiones y respuestas son JSON.

1.1. Códigos de respuesta

Cada ficha de ruta lista **solo** los códigos que esa ruta puede devolver según el código de su router, del esquema Pydantic y de los triggers. Su significado es el siguiente.

Código	Significado en esta API
200	*OK.* Consulta o actualización correcta (GET, PUT, PATCH y el check-in).
201	*Created.* Registro creado (POST de creación).
204	*No Content.* Registro eliminado; la respuesta no tiene cuerpo (DELETE).
404	*Not Found.* El router comprueba la existencia del recurso y no existe.
409	*Conflict.* La operación fue rechazada: o bien por un trigger de la base (el mensaje de la regla viaja en `detail`), o bien por un mensaje propio del router.
422	*Unprocessable Entity.* El cuerpo o los parámetros no pasan la validación de FastAPI/Pydantic, o el router valida explícitamente (`salones/disponibles`).

1.2. Mensajes de los triggers en el 409

Cuando un trigger bloquea una operación, el router devuelve 409 con el texto de la excepción de PostgreSQL. Ese texto empieza por un prefijo que identifica la regla; el frontend lo usa para decir «Operación bloqueada por la regla A*n*».

Prefijo	**Regla**	**Qué comprueba**
`SALON INACTIVO`	A1	El salón está deshabilitado por mantenimiento.
`CAPACIDAD`	A1	El aforo esperado supera la capacidad del salón.
`DISPONIBILIDAD`	A1	El horario se cruza con otro evento no cancelado del mismo salón; debe haber 1 hora libre entre eventos.
`CUPO`	A1 y A2	A1: al reprogramar, el aforo no puede quedar por debajo de los inscritos. A2: el evento ya alcanzó su aforo contratado.
`INSCRIPCION`	A2	El evento está Cancelado o Finalizado y no recibe inscripciones.
`CHECK-IN`	A2	Llegada ya registrada, evento que no está Confirmado, o hora fuera de la ventana (desde 1 hora antes del inicio hasta el fin).
`SERVICIOS`	A3	Evento Finalizado o Cancelado (sus servicios no se modifican), o servicio cobrado por Unidad sin cantidad.
`ESTADO`	A4	Transición no permitida, o evento Finalizado/Cancelado que ya no admite cambios.
`CIERRE`	A4	Se intenta finalizar un evento antes de su hora de fin.

Dos rutas devuelven además 409 con mensajes del propio router (no de un trigger): el registro de clientes y la inscripción de asistentes. Se indican en sus fichas.

1.3. Casos no controlados por la API (500)

Algunos routers solo capturan los errores de trigger (`InternalError`) y no los de integridad. Cuando ocurre uno de estos casos, la API responde 500 y el frontend muestra «Error 500: Error interno del servidor.». Este manual los señala en la ficha de cada ruta afectada; salen de la lectura del código y no se ejecutaron para documentarlos:

- Registrar un asistente con un documento ya existente (restricción UNIQUE).
- Registrar staff o crear un evento con una llave foránea inexistente (`jefe_id`, `salon_id`, `cliente_id`, `coordinador_id`).
- Agregar a un evento un servicio que ya tiene contratado (UNIQUE evento y servicio).
- Modificar o quitar un servicio de un evento Finalizado o Cancelado: el trigger A3 bloquea la operación, pero esas dos rutas no capturan la excepción.

1.4. Evidencia de pruebas del Entregable 1

Las pruebas MIN-01 a MIN-23 se ejecutaron en el Entregable 1 con Postman y están documentadas en *Reporte Evidencia Funcionamiento.pdf* (capítulos 4 a 12, un apartado por prueba). Los reportes B1 a B4 y las reglas A4 y A5 tienen sus capturas en la carpeta de evidencia del Entregable 1 (`documentacion_back/` en el repositorio), con nombres como «B1 Reporte Organigrama» o «A5-01 Consultar salones disponibles para horario y aforo». Este manual solo *remite* a esa evidencia: no la reproduce ni la ejecuta de nuevo.

1.5. Encabezado X-Usuario

El frontend envía el encabezado `X-Usuario` en todas las peticiones (valor del campo «Usuario de la aplicación»). La API actual no lo lee: la auditoría registra el usuario de la conexión a la base de datos.

2. Resumen de rutas

Las 27 rutas de la tabla las usa el frontend; la columna de funciones se verificó en `front/api_client.py`. La última columna da el apartado de *Reporte Evidencia Funcionamiento.pdf* donde está la prueba (los reportes B1–B4 no están en ese PDF; ver la sección 1.4).

Prueba	Método	Ruta	Función en `api_client.py`	Apartado
MIN-01	GET	`/clientes/`	`get_clientes`	4.1
MIN-02	POST	`/clientes/`	`crear_cliente`	4.2
MIN-03	GET	`/clientes/{id}/eventos`	`get_eventos_cliente`	4.3
MIN-04	GET	`/staff/`	`get_staff`	5.1
MIN-05	POST	`/staff/`	`crear_staff`	5.2
MIN-06	GET	`/salones/`	`get_salones`	6.1
MIN-07	GET	`/servicios/`	`get_servicios`	6.2
MIN-08	GET	`/salones/disponibles`	`get_salones_disponibles`	6.3
MIN-09	GET	`/eventos/`	`get_eventos`	7.1
MIN-10	POST	`/eventos/`	`crear_evento`	7.2
MIN-11	GET	`/eventos/{id}`	`get_evento`	7.3
MIN-12	PATCH	`/eventos/{id}`	`reprogramar_evento`	7.4
MIN-13	PATCH	`/eventos/{id}/estado`	`cambiar_estado`	7.5
MIN-14	POST	`/eventos/{id}/servicios`	`agregar_servicio`	8.1
MIN-15	PUT	`/eventos/{id}/servicios`	`modificar_servicio`	8.2
MIN-16	DELETE	`/eventos/{id}/servicios`	`quitar_servicio`	8.3
MIN-17	GET	`/eventos/{id}/cotizacion`	`get_cotizacion`	9.1
MIN-18	GET	`/eventos/{id}/auditoria`	`get_auditoria`	10.1
MIN-19	GET	`/asistentes/`	`get_asistentes`	11.1
MIN-20	POST	`/asistentes/`	`crear_asistente`	11.2
MIN-21	POST	`/eventos/{id}/inscripciones`	`inscribir`	12.1
MIN-22	GET	`/eventos/{id}/inscripciones`	`get_inscripciones`	12.2
MIN-23	POST	`/eventos/{id}/checkin`	`checkin`	12.3
B1	GET	`/reportes/organigrama`	`reporte_organigrama`	–
B2	GET	`/reportes/escalamiento/{id}`	`reporte_escalamiento`	–
B3	GET	`/reportes/pareto-clientes`	`reporte_pareto`	–
B4	GET	`/reportes/ocupacion-salones`	`reporte_ocupacion`	–

3. Clientes

Un cliente es una empresa (NIT) o una persona natural (CC). El número de documento no se repite.

MIN-01. Listar clientes

Ruta	**GET** `/clientes/`
Propósito	Devuelve los clientes registrados, con búsqueda opcional.
Parámetros o cuerpo	Parámetros de consulta, todos opcionales: `buscar` (texto; coincide de forma parcial y sin distinguir mayúsculas con nombre, número de documento o correo) y `tipo_documento` (`CC` o `NIT`, igualdad exacta).
Respuestas	**200** Lista de clientes: `id`, `tipo_documento`, `numero_documento`, `nombre`, `email`, `telefono`, `fecha_registro`. Puede estar vacía. No define otros códigos.
Uso en el frontend	`get_clientes(buscar)`; solo envía `buscar`. Pantallas: Clientes y staff → Clientes; selector de cliente en Eventos.
Evidencia previa	Prueba MIN-01; ver *Reporte Evidencia Funcionamiento.pdf*, sección 4.1.

MIN-02. Registrar cliente

Ruta	**POST** `/clientes/`
Propósito	Crea un cliente nuevo.
Parámetros o cuerpo	Cuerpo `ClienteCreate`, todos los campos obligatorios: `tipo_documento` (`CC` o `NIT`); `numero_documento` (1 a 10 dígitos); `nombre` (1 a 60 caracteres); `email` (formato de correo válido); `telefono` (7 a 10 dígitos).
Respuestas	**201** Cliente creado, con `id` y `fecha_registro`. **409** Mensaje del router: «Ya existe un cliente con ese número de documento.» Se devuelve ante cualquier error de integridad al insertar (documento repetido u otra restricción CHECK de la tabla); no viene de un trigger. **422** El cuerpo no cumple `ClienteCreate`.
Uso en el frontend	`crear_cliente`. Clientes y staff → Registrar cliente (valida antes con el mismo esquema).
Evidencia previa	Prueba MIN-02; ver *Reporte Evidencia Funcionamiento.pdf*, sección 4.2.

MIN-03. Consultar eventos de un cliente

Ruta	**GET** `/clientes/{id}/eventos`
Propósito	Devuelve los eventos que pertenecen a un cliente.
Parámetros o cuerpo	Parámetro de ruta `id`: entero, identificador del cliente.
Respuestas	**200** Lista de eventos del cliente (campos de `EventoResponse`); vacía si no tiene eventos. **404** «Cliente no encontrado». **422** `id` no es un entero.
Uso en el frontend	`get_eventos_cliente`. Clientes y staff → Clientes → «Eventos del cliente».
Evidencia previa	Prueba MIN-03; ver *Reporte Evidencia Funcionamiento.pdf*, sección 4.3.

4. Staff

El staff son los empleados del centro; cada persona, salvo el Director, tiene un jefe inmediato (`jefe_id`).

MIN-04. Listar staff

Ruta	**GET** `/staff/`
Propósito	Devuelve todo el personal registrado.
Parámetros o cuerpo	Sin parámetros.
Respuestas	**200** Lista con `id`, `nombre`, `cargo`, `area`, `email`, `fecha_ingreso` y `jefe_id` (nulo en el Director). No define otros códigos.
Uso en el frontend	`get_staff`. Clientes y staff → Staff; coordinador en Crear evento; selectores de Reportes.
Evidencia previa	Prueba MIN-04; ver *Reporte Evidencia Funcionamiento.pdf*, sección 5.1.

MIN-05. Registrar staff

Ruta	**POST** `/staff/`
Propósito	Crea un miembro del staff.
Parámetros o cuerpo	Cuerpo `StaffCreate`: `nombre` (2 a 60 caracteres); `cargo` (`Director`, `Gerente`, `Jefe`, `Coordinador`, `Auxiliar` o `Técnico`); `area` (`Dirección`, `Operaciones`, `Comercial`, `Logística`, `Alimentos y Bebidas` o `Audiovisuales`); `email` (correo válido); `fecha_ingreso` (fecha); `jefe_id` (entero mayor que 0, opcional). Regla del esquema: el Director no puede tener jefe y los demás cargos deben tenerlo.
Respuestas	**201** Staff creado, con su `id`. **422** El cuerpo no cumple `StaffCreate`, incluida la regla del jefe: «El director no puede tener jefe asignado.» o «El miembro del Staff debe tener un jefe asignado.»
Uso en el frontend	`crear_staff`. Clientes y staff → Registrar staff (valida antes con el mismo esquema).
Evidencia previa	Prueba MIN-05; ver *Reporte Evidencia Funcionamiento.pdf*, sección 5.2.

5. Catálogo y disponibilidad

MIN-06. Listar salones

Ruta	**GET** `/salones/`
Propósito	Devuelve el catálogo de salones.
Parámetros o cuerpo	Sin parámetros.
Respuestas	**200** Lista con `id`, `nombre_salon`, `tamano` (`Pequeño`, `Mediano` o `Grande`), `capacidad`, `precio_hora` y `habilitado`. No define otros códigos.
Uso en el frontend	`get_salones`. Nombres de salón en Inicio y Eventos; lista de salones al Reprogramar.
Evidencia previa	Prueba MIN-06; ver *Reporte Evidencia Funcionamiento.pdf*, sección 6.1.

MIN-07. Listar servicios

Ruta	**GET** `/servicios/`
Propósito	Devuelve el catálogo de servicios (bebidas, refrigerios y audiovisuales).
Parámetros o cuerpo	Sin parámetros.
Respuestas	**200** Lista con `id`, `nombre_servicio`, `descripcion`, `unidad_cobro` (`Hora`, `Unidad` o `Persona`) y `precio`. No define otros códigos.
Uso en el frontend	`get_servicios`. Eventos → Gestionar evento → Servicios.
Evidencia previa	Prueba MIN-07; ver *Reporte Evidencia Funcionamiento.pdf*, sección 6.2.

MIN-08. Consultar salones disponibles

Ruta	**GET** `/salones/disponibles`
Propósito	Llama a la función A5 (`fn_salones_disponibles`): salones habilitados, con capacidad suficiente y sin cruce de horario (incluida 1 hora de montaje), con el costo estimado para ese horario.
Parámetros o cuerpo	Parámetros de consulta, los tres obligatorios: `fecha_inicio` y `fecha_fin` (fecha y hora, formato ISO) y `aforo` (entero mayor que 0).
Respuestas	**200** Lista ordenada de menor a mayor capacidad con `salon_id`, `nombre_salon`, `tamano`, `capacidad`, `precio_hora` y `costo_estimado` (horas del evento redondeadas hacia arriba por el precio por hora). Puede estar vacía. **422** Falta un parámetro, no tiene el tipo esperado o `aforo` es menor o igual a 0 (FastAPI); o `fecha_fin` no es posterior a `fecha_inicio` (el router responde «La fecha y hora de fin debe ser posterior al inicio»). No hay 404 ni 409: es una consulta.
Uso en el frontend	`get_salones_disponibles`. Disponibilidad de salones; selector de salón en Eventos → Crear evento.
Evidencia previa	Prueba MIN-08; ver *Reporte Evidencia Funcionamiento.pdf*, sección 6.3. Regla A5: capturas A5-01 a A5-05 del Entregable 1.

6. Eventos

El evento es la entidad central. Nace en estado Cotizado y sigue el ciclo Cotizado → Confirmado → Finalizado, o Cancelado desde Cotizado o Confirmado.

MIN-09. Listar eventos

Ruta	**GET** `/eventos/`
Propósito	Devuelve los eventos, con filtros opcionales.
Parámetros o cuerpo	Parámetros de consulta, todos opcionales: `estado` (texto; no se valida: un valor desconocido devuelve lista vacía), `cliente_id` y `salon_id` (enteros).
Respuestas	**200** Lista de eventos: `id`, `nombre`, `tipo`, `inicio`, `fin`, `aforo_esperado`, `salon_id`, `cliente_id`, `coordinador_id`, `estado`, `precio_hora_salon_cotizado`, `valor_total_cotizado` y `tasa_asistencia`. **422** `cliente_id` o `salon_id` no es un entero.
Uso en el frontend	`get_eventos(estado)`; solo envía `estado`. Inicio (tablero), Eventos → Listar y Gestionar, selectores de Asistentes y check-in.
Evidencia previa	Prueba MIN-09; ver *Reporte Evidencia Funcionamiento.pdf*, sección 7.1.

MIN-10. Crear evento

Ruta	**POST** `/eventos/`
Propósito	Crea un evento. La base le asigna el estado Cotizado, congela el precio por hora del salón y calcula el valor total (A3: horas redondeadas hacia arriba por el precio por hora; aún sin servicios).
Parámetros o cuerpo	Cuerpo `EventoCreate`, todos obligatorios: `nombre` (1 a 100 caracteres); `tipo` (`Congreso`, `Conferencia`, `Taller`, `Feria`, `Seminario`, `Exposición`, `Corporativo`, `Social` u `Otro`); `inicio` y `fin` (fecha y hora; el fin debe ser posterior al inicio); `aforo_esperado` (entero mayor que 0); `salon_id`, `cliente_id` y `coordinador_id` (enteros mayores que 0).
Respuestas	**201** Evento creado (`EventoResponse`) con `estado` Cotizado y `valor_total_cotizado` ya calculado. **409** Regla A1 (trigger): `SALON INACTIVO`, `CAPACIDAD` o `DISPONIBILIDAD`. Ejemplo: «DISPONIBILIDAD: El salón "…" se cruza con el evento N "…" (… a …). Se requiere 1 hora libre entre eventos.» **422** El cuerpo no cumple `EventoCreate` (incluye «La fecha y hora de fin debe ser posterior al inicio.»).
Uso en el frontend	`crear_evento`. Eventos → Crear evento (valida antes con `EventoCreate` y solo ofrece salones devueltos por MIN-08).
Evidencia previa	Prueba MIN-10; ver *Reporte Evidencia Funcionamiento.pdf*, sección 7.2.

MIN-11. Obtener detalle de evento

Ruta	**GET** `/eventos/{id}`
Propósito	Devuelve un evento con sus servicios contratados y su resumen de asistencia.
Parámetros o cuerpo	Parámetro de ruta `id`: entero, identificador del evento.
Respuestas	**200** Los campos del evento más `servicios` (lista de líneas contratadas) y `asistencia` (`inscritos`, `asistentes_reales`, `tasa_asistencia`). **404** «Evento no encontrado». **422** `id` no es un entero.
Uso en el frontend	`get_evento`. Eventos → Gestionar evento (resumen, métricas y pestaña Servicios).
Evidencia previa	Prueba MIN-11; ver *Reporte Evidencia Funcionamiento.pdf*, sección 7.3.

MIN-12. Reprogramar evento

Ruta	**PATCH** `/eventos/{id}`
Propósito	Cambia horario, aforo o salón de un evento. A1 revalida disponibilidad y capacidad, A3 recalcula el total y A4 registra el cambio en la auditoría.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo `EventoUpdate`, todos los campos opcionales: `inicio`, `fin` (si llegan ambos, el fin debe ser posterior al inicio), `aforo_esperado` (mayor que 0) y `salon_id` (mayor que 0).
Respuestas	**200** Evento actualizado (`EventoResponse`). **404** «Evento no encontrado». **409** Trigger A1: `SALON INACTIVO`, `CAPACIDAD`, `DISPONIBILIDAD` o `CUPO` («El evento "…" ya tiene N inscritos; el aforo no puede bajar a M.»). Trigger A4: `ESTADO` si el evento ya está Finalizado o Cancelado («… ya no admite cambios»). **422** El cuerpo no cumple `EventoUpdate`; por ejemplo, fin anterior o igual al inicio, o `id` no entero.
Uso en el frontend	`reprogramar_evento`. Eventos → Gestionar evento → Reprogramar (envía inicio, fin, aforo y salón).
Evidencia previa	Prueba MIN-12; ver *Reporte Evidencia Funcionamiento.pdf*, sección 7.4.

MIN-13. Cambiar estado de evento

Ruta	**PATCH** `/eventos/{id}/estado`
Propósito	Confirma, finaliza o cancela un evento. El trigger A4 controla las transiciones, calcula la tasa de asistencia al finalizar y registra el cambio en la auditoría.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo: `estado` (`Cotizado`, `Confirmado`, `Finalizado` o `Cancelado`). Transiciones permitidas: Cotizado → Confirmado o Cancelado; Confirmado → Finalizado o Cancelado. Finalizado y Cancelado son definitivos.
Respuestas	**200** Evento con el nuevo estado. Al finalizar, `tasa_asistencia` = llegadas / inscritos × 100 (nula si no hay inscritos). **404** «Evento no encontrado». **409** Trigger A4: `ESTADO` («No se puede pasar un evento de X a Y.» o «El evento "…" está Finalizado/Cancelado y ya no admite cambios.») o `CIERRE` («El evento "…" termina el AAAA-MM-DD HH:MM y solo se puede finalizar después de esa hora.»). **422** `estado` no es uno de los cuatro valores, o `id` no es entero.
Uso en el frontend	`cambiar_estado`. Eventos → Gestionar evento → Estado (ofrece Confirmado, Finalizado y Cancelado).
Evidencia previa	Prueba MIN-13; ver *Reporte Evidencia Funcionamiento.pdf*, sección 7.5. Regla A4: capturas A4-01 a A4-07 del Entregable 1.

7. Servicios por evento

La tabla `servicios_eventos` relaciona eventos y servicios; al contratar un servicio se copia su precio (precio congelado). Las tres rutas comparten la ruta `/eventos/{id}/servicios` y se distinguen por el método.

MIN-14. Agregar servicio a evento

Ruta	**POST** `/eventos/{id}/servicios`
Propósito	Contrata un servicio del catálogo para el evento. El trigger A3 congela el precio, completa la cantidad si no se envía y recalcula el valor total del evento.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo: `servicio_id` (entero mayor que 0, obligatorio) y `cantidad` (entero mayor que 0, opcional). Si no se envía la cantidad: servicio por Persona, usa el aforo; por Hora, las horas del evento (hacia arriba); por Unidad, la cantidad es obligatoria.
Respuestas	**201** Línea creada: `id`, `evento_id`, `servicio_id`, `precio_unidad_cotizado` y `cantidad`. **404** «Evento no encontrado». **409** Trigger A3, prefijo `SERVICIOS`: el evento está Finalizado o Cancelado, o el servicio cobrado por Unidad se envió sin cantidad. **422** El cuerpo no cumple el esquema, o `id` no es entero.
Uso en el frontend	`agregar_servicio`. Eventos → Gestionar evento → Servicios (cantidad 0 en pantalla se envía como «sin cantidad»).
Evidencia previa	Prueba MIN-14; ver *Reporte Evidencia Funcionamiento.pdf*, sección 8.1.

MIN-15. Actualizar servicio de evento

Ruta	**PUT** `/eventos/{id}/servicios`
Propósito	Modifica la cantidad de un servicio ya contratado. El precio congelado no cambia y A3 recalcula el total.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo `ServicioEventoUpdate`: `servicio_id` (identifica la línea; el esquema lo marca opcional, pero sin él no se encuentra ninguna) y `cantidad` (entero mayor que 0).
Respuestas	**200** Línea actualizada (`ServicioEventoResponse`). **404** «Servicio no encontrado en este evento» (no hay línea con ese evento y servicio). **422** El cuerpo no cumple el esquema, o `id` no es entero.
Uso en el frontend	`modificar_servicio`. Eventos → Gestionar evento → Servicios, botón «Modificar cantidad».
Evidencia previa	Prueba MIN-15; ver *Reporte Evidencia Funcionamiento.pdf*, sección 8.2.

MIN-16. Eliminar servicio de evento

Ruta	**DELETE** `/eventos/{id}/servicios`
Propósito	Quita un servicio contratado del evento. A3 recalcula el valor total.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo JSON (inusual en DELETE): `servicio_id` (entero mayor que 0).
Respuestas	**204** Línea eliminada; sin cuerpo. **404** «Servicio no encontrado en este evento». **422** El cuerpo no cumple el esquema, o `id` no es entero.
Uso en el frontend	`quitar_servicio`. Eventos → Gestionar evento → Servicios, botón «Quitar servicio».
Evidencia previa	Prueba MIN-16; ver *Reporte Evidencia Funcionamiento.pdf*, sección 8.3.

8. Cotización y auditoría

MIN-17. Consultar cotización del evento

Ruta	**GET** `/eventos/{id}/cotizacion`
Propósito	Desglose del costo: alquiler del salón más servicios contratados con su precio congelado.
Parámetros o cuerpo	Parámetro de ruta `id`: entero.
Respuestas	**200** `evento_id`, `hora_salon` (horas hacia arriba), `precio_hora_salon`, `valor_total_salon`, `servicios` (nombre, unidad de cobro, precio unitario, cantidad y subtotal), `total_servicios` y `valor_total_cotizado`. **404** «Evento no encontrado». **422** `id` no es un entero.
Uso en el frontend	`get_cotizacion`. Eventos → Gestionar evento → Cotización.
Evidencia previa	Prueba MIN-17; ver *Reporte Evidencia Funcionamiento.pdf*, sección 9.1.

MIN-18. Consultar auditoría del evento

Ruta	**GET** `/eventos/{id}/auditoria`
Propósito	Historial de cambios de estado, salón, inicio y fin que registra el trigger A4.
Parámetros o cuerpo	Parámetro de ruta `id`: entero.
Respuestas	**200** Lista ordenada por fecha ascendente: `id`, `evento_id`, `campo_modificado`, `valor_anterior`, `valor_nuevo`, `fecha_hora` y `usuario`. Puede estar vacía. **404** «Evento no encontrado». **422** `id` no es un entero.
Uso en el frontend	`get_auditoria`. Eventos → Gestionar evento → Auditoría.
Evidencia previa	Prueba MIN-18; ver *Reporte Evidencia Funcionamiento.pdf*, sección 10.1.

9. Asistentes

MIN-19. Listar asistentes

Ruta	**GET** `/asistentes/`
Propósito	Devuelve las personas registradas como posibles asistentes.
Parámetros o cuerpo	Sin parámetros.
Respuestas	**200** Lista con `id`, `documento_identidad`, `nombre`, `email` y `empresa`. No define otros códigos.
Uso en el frontend	`get_asistentes`. Asistentes y check-in (selector de Inscribir, datos de Inscritos y llegadas, tabla de Registrar asistente).
Evidencia previa	Prueba MIN-19; ver *Reporte Evidencia Funcionamiento.pdf*, sección 11.1.

MIN-20. Registrar asistente

Ruta	**POST** `/asistentes/`
Propósito	Crea un asistente. El documento es único.
Parámetros o cuerpo	Cuerpo `AsistenteCreate`, todos obligatorios: `documento_identidad` (1 a 10 dígitos); `nombre` (2 a 60 caracteres); `email` (correo válido); `empresa` (1 a 100 caracteres).
Respuestas	**201** Asistente creado, con su `id`. **422** El cuerpo no cumple `AsistenteCreate`.
Uso en el frontend	`crear_asistente`. Asistentes y check-in → Registrar asistente (valida antes con el mismo esquema).
Evidencia previa	Prueba MIN-20; ver *Reporte Evidencia Funcionamiento.pdf*, sección 11.2.

10. Inscripciones y check-in

MIN-21. Inscribir asistente en evento

Ruta	**POST** `/eventos/{id}/inscripciones`
Propósito	Inscribe un asistente en un evento. A2 impide inscribir en eventos Cancelados o Finalizados y por encima del aforo.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo: `asistente_id` (entero mayor que 0).
Respuestas	**201** Inscripción creada: `id`, `asistente_id`, `evento_id` y `fecha_checkin` nula. **404** «Evento no encontrado». **409** Mensaje del router: «El asistente ya está inscrito en este evento» (si ya existe la inscripción, y también ante cualquier error de integridad, como un `asistente_id` inexistente). Trigger A2: `INSCRIPCION` (evento Cancelado o Finalizado) o `CUPO` («El evento "…" alcanzó su aforo contratado (N personas).»). **422** El cuerpo no cumple el esquema, o `id` no es entero.
Uso en el frontend	`inscribir`. Asistentes y check-in → Inscribir.
Evidencia previa	Prueba MIN-21; ver *Reporte Evidencia Funcionamiento.pdf*, sección 12.1.

MIN-22. Listar inscripciones de un evento

Ruta	**GET** `/eventos/{id}/inscripciones`
Propósito	Devuelve los inscritos de un evento y si ya hicieron check-in.
Parámetros o cuerpo	Parámetro de ruta `id`: entero.
Respuestas	**200** Lista de inscripciones (`id`, `asistente_id`, `evento_id`, `fecha_checkin`); puede estar vacía. **404** «Evento no encontrado». **422** `id` no es un entero.
Uso en el frontend	`get_inscripciones`. Asistentes y check-in → Inscritos y llegadas.
Evidencia previa	Prueba MIN-22; ver *Reporte Evidencia Funcionamiento.pdf*, sección 12.2.

MIN-23. Registrar check-in

Ruta	**POST** `/eventos/{id}/checkin`
Propósito	Registra la llegada de un asistente a partir de su documento. La hora la toma la base de datos (`now()`); A2 valida estado, ventana horaria y unicidad.
Parámetros o cuerpo	Parámetro de ruta `id`. Cuerpo: `documento_identidad` (1 a 10 dígitos).
Respuestas	**200** Inscripción con `fecha_checkin` ya registrada (esta ruta responde 200, no 201). **404** Tres mensajes posibles: «Evento no encontrado»; «Asistente no encontrado» (ningún asistente con ese documento); «El asistente no está inscrito en este evento». **409** Trigger A2, prefijo `CHECK-IN`: «Este asistente ya registró su llegada el . . . »; «El evento "..." está X; solo se registra llegada en eventos confirmados.»; o «Solo se permite ingresar entre . . . y . . . (hora indicada: . . .).» (ventana: desde 1 hora antes del inicio hasta el fin). **422** `documento_identidad` vacío o con caracteres no numéricos, o `id` no entero.
Uso en el frontend	`checkin`. Asistentes y check-in → Check-in (no valida el documento antes de enviarlo).
Evidencia previa	Prueba MIN-23; ver *Reporte Evidencia Funcionamiento.pdf*, sección 12.3.

11. Reportes B1 a B4

Los reportes se calculan en la base (funciones y vistas de `back/sql/`); son de solo lectura y por eso ninguno puede devolver 409. Ninguna de sus pruebas está en *Reporte Evidencia Funcionamiento.pdf*: la evidencia del Entregable 1 son las capturas B1 a B4 de `documentacion_back/`.

B1. Organigrama

Ruta	**GET** `/reportes/organigrama`
Propósito	Jerarquía del staff con una consulta recursiva (CTE): nivel, ruta de mando y personas a cargo.
Parámetros o cuerpo	Parámetro de consulta opcional: `staff_id` (entero mayor que 0). Sin él, el organigrama completo desde quien no tiene jefe; con él, el árbol desde esa persona.
Respuestas	**200** Lista con `staff_id`, `nombre`, `cargo`, `area`, `jefe_id`, `nivel`, `ruta_mando` y `personas_a_cargo`. Un `staff_id` inexistente devuelve lista vacía. **422** `staff_id` es menor o igual a 0 o no es entero.
Uso en el frontend	`reporte_organigrama`; envía `staff_id` solo si se elige una persona. Reportes → B1.
Evidencia previa	Captura «B1 Reporte Organigrama» (Entregable 1, `documentacion_back/`).

B2. Cadena de escalamiento

Ruta	**GET** `/reportes/escalamiento/{id}`
Propósito	Cadena de mando ascendente desde un miembro del staff hasta el Director, con consulta recursiva.
Parámetros o cuerpo	Parámetro de ruta `id`: entero, identificador del staff.
Respuestas	**200** Lista ordenada: `orden` (1 es el jefe inmediato), `staff_id`, `nombre`, `cargo`, `area` y `email`. **404** «No se encontró cadena de escalamiento para este miembro del staff»: el staff no existe o no tiene jefe (el caso del Director). **422** `id` no es un entero.
Uso en el frontend	`reporte_escalamiento`. Reportes → B2 (el selector solo lista personas con jefe).
Evidencia previa	Captura «B2 Reporte Escalamiento» (Entregable 1, `documentacion_back/`).

B3. Ranking de clientes (Pareto)

Ruta	**GET** `/reportes/pareto-clientes`
Propósito	Ranking de clientes por total facturado, con participación y porcentaje acumulado (funciones de ventana). Solo cuentan eventos no cancelados.
Parámetros o cuerpo	Sin parámetros.
Respuestas	**200** Lista ordenada por `posicion`: `cliente_id`, `cliente`, `eventos`, `total_facturado`, `participacion`, `acumulado` y `segmento_pareto` (`A` mientras el acumulado anterior al cliente sea menor a 80 %; `B` en adelante). No define otros códigos.
Uso en el frontend	`reporte_pareto`. Reportes → B3.
Evidencia previa	Captura «B3 Reporte Pareto Clientes» (Entregable 1, `documentacion_back/`).

B4. Ocupación de salones

Ruta	**GET** `/reportes/ocupacion-salones`
Propósito	Eventos por salón en orden cronológico con evento anterior, días libres entre eventos y horas acumuladas (`ROW_NUMBER`, `LAG` y suma acumulada). Solo eventos no cancelados.
Parámetros o cuerpo	Parámetro de consulta opcional: `salon_id` (entero mayor que 0) para ver un solo salón.
Respuestas	**200** Lista con `salon_id`, `salon`, `evento_id`, `evento`, `inicio`, `fin`, `numero_en_salon`, `evento_anterior`, `dias_libres` y `horas_acumuladas`; el primer evento de cada salón no tiene anterior ni días libres. **422** `salon_id` es menor o igual a 0 o no es entero.
Uso en el frontend	`reporte_ocupacion`; no envía `salon_id`. Reportes → B4.
Evidencia previa	Captura «B4 Reporte Ocupación Salones» (Entregable 1, `documentacion_back/`).